C++ Monitoring Module

```
#ifndef MONITOR_H
#define MONITOR_H

#ifndef _WIN32_WINNT
#define _WIN32_WINNT 0x0600
#endif
#ifndef WINVER
#define WINVER 0x0600
#endif

#include <windows.h>
#include <psapi.h>
#include <tlhelp32.h>
#include <string>
#include <vector>
#include <cstdint>

struct SystemMetrics {
    double cpu_percent;
    double ram_used_mb;
    double ram_total_mb;
    double ram_percent;
    int process_count;
    uint64_t uptime_seconds;
    std::string timestamp;
};

struct ProcessInfo {
    uint32_t pid;
    std::string name;
    double memory_mb;
    int thread_count;
    std::string priority_class;
    std::string subsystem; // e.g. "Navigation", "Audio Media", "Safety
Camera", "CAN-Bus", "HVAC"
```

```
};

struct VehicleTelemetry {
    int speed_kmh;
    int rpm;
    int battery_percent;
    double cabin_temp_c;
    std::string gear; // "P", "R", "N", "D"
    bool reverse_camera_active;
    std::string driving_mode; // "Comfort", "Sport", "Eco"
    double bus_load_percent;
    double coolant_temp_c;
};

class SystemMonitor {
public:
    SystemMonitor();
    ~SystemMonitor();

    void initialize();
    SystemMetrics getSystemMetrics();
    std::vector<ProcessInfo> getProcessList(size_t limit = 100);
    std::vector<ProcessInfo> getIviProcessList();
    VehicleTelemetry getVehicleTelemetry();
    void setVehicleGear(const std::string& gear);
    void setVehicleMode(const std::string& mode);

    static std::string priorityClassToString(DWORD priorityClass);
    static DWORD stringToPriorityClass(const std::string& prioStr);

private:
    FILETIME prevIdleTime;
    FILETIME prevKernelTime;
    FILETIME prevUserTime;
    bool hasPreviousTimes;

    VehicleTelemetry vehicleState;
    std::vector<ProcessInfo> iviProcesses;

    static uint64_t fileTimeToUInt64(const FILETIME& ft);
```

```
    double calculateCpuUsage();
    std::string getCurrentIsoTimestamp();
    void updateSimulatedVehiclePhysics();
};

#endif // MONITOR_H
```

Monitoring.cpp

```
#include "monitor.h"
#include <iostream>
#include <iomanip>
#include <sstream>
#include <algorithm>
#include <ctime>
#include <cmath>

SystemMonitor::SystemMonitor() : hasPreviousTimes(false) {
    prevIdleTime = {0, 0};
    prevKernelTime = {0, 0};
    prevUserTime = {0, 0};

    // Initialize Automotive Infotainment (IVI) state
    vehicleState.speed_kmh = 64;
    vehicleState.rpm = 2150;
    vehicleState.battery_percent = 86;
    vehicleState.cabin_temp_c = 21.5;
    vehicleState.gear = "D";
    vehicleState.reverse_camera_active = false;
    vehicleState.driving_mode = "Comfort";
    vehicleState.bus_load_percent = 26.8;
    vehicleState.coolant_temp_c = 88.5;

    // Core Automotive IVI Subsystems (including Google Maps & Spotify)
    iviProcesses = {
        {1042, "google_maps_nav.bin", 438.5, 24, "ABOVE_NORMAL",
"Navigation (Google Maps GPS)"},
        {1088, "spotify_audio_dsp.bin", 194.2, 16, "NORMAL", "Audio Media
(Spotify Stream)"},
        {1005, "rear_camera_feed.bin", 88.0, 8, "REALTIME", "Safety
(Rear-View HD Stream)"},
```

```
        {1010, "can_bus_gateway.bin", 42.1, 6, "HIGH", "CAN-Bus Telemetry
Gateway"},
        {1060, "hvac_climate_ui.bin", 96.4, 10, "NORMAL", "HVAC Cabin
Climate Control"},
        {1018, "instrument_cluster_sync.bin", 64.8, 12, "HIGH", "Virtual
Cockpit & HUD Sync"},
        {1120, "ota_telematics_sync.bin", 112.0, 8, "IDLE", "Cloud
Diagnostics & OTA Sync"}
    };
}

SystemMonitor::~SystemMonitor() {}

void SystemMonitor::initialize() {
    GetSystemTimes(&prevIdleTime, &prevKernelTime, &prevUserTime);
    hasPreviousTimes = true;
}

uint64_t SystemMonitor::fileTimeToUInt64(const FILETIME& ft) {
    ULARGE_INTEGER uli;
    uli.LowPart = ft.dwLowDateTime;
    uli.HighPart = ft.dwHighDateTime;
    return uli.QuadPart;
}

std::string SystemMonitor::getCurrentIsoTimestamp() {
    SYSTEMTIME st;
    GetLocalTime(&st);
    char buf[64];
    snprintf(buf, sizeof(buf), "%04d-%02d-%02d %02d:%02d:%02d",
             st.wYear, st.wMonth, st.wDay, st.wHour, st.wMinute,
st.wSecond);
    return std::string(buf);
}

void SystemMonitor::updateSimulatedVehiclePhysics() {
    // Dynamic automotive physics simulation
    if (vehicleState.gear == "D") {
        vehicleState.speed_kmh = 60 + (rand() % 12);
        vehicleState.rpm = 2100 + (rand() % 250);
```

```
        vehicleState.reverse_camera_active = false;
        vehicleState.bus_load_percent = 26.0 + (rand() % 8);
    } else if (vehicleState.gear == "R") {
        vehicleState.speed_kmh = 10 + (rand() % 5);
        vehicleState.rpm = 1200 + (rand() % 150);
        vehicleState.reverse_camera_active = true;
        vehicleState.bus_load_percent = 38.5; // High bus load from video
frames
    } else if (vehicleState.gear == "P" || vehicleState.gear == "N") {
        vehicleState.speed_kmh = 0;
        vehicleState.rpm = 820 + (rand() % 40);
        vehicleState.reverse_camera_active = false;
        vehicleState.bus_load_percent = 14.2;
    }
}

VehicleTelemetry SystemMonitor::getVehicleTelemetry() {
    updateSimulatedVehiclePhysics();
    return vehicleState;
}

void SystemMonitor::setVehicleGear(const std::string& gear) {
    vehicleState.gear = gear;
    if (gear == "R") {
        vehicleState.reverse_camera_active = true;
        // Boost rear camera to REALTIME
        for (auto& p : iviProcesses) {
            if (p.name == "rear_camera_feed.bin") p.priority_class =
"REALTIME";
        }
    } else {
        vehicleState.reverse_camera_active = false;
    }
}

void SystemMonitor::setVehicleMode(const std::string& mode) {
    vehicleState.driving_mode = mode;
}

double SystemMonitor::calculateCpuUsage() {
```

```
    FILETIME idleTime, kernelTime, userTime;
    if (!GetSystemTimes(&idleTime, &kernelTime, &userTime)) {
        return 0.0;
    }

    if (!hasPreviousTimes) {
        prevIdleTime = idleTime;
        prevKernelTime = kernelTime;
        prevUserTime = userTime;
        hasPreviousTimes = true;
        return 0.0;
    }

    uint64_t idle = fileTimeToUInt64(idleTime) -
fileTimeToUInt64(prevIdleTime);
    uint64_t kernel = fileTimeToUInt64(kernelTime) -
fileTimeToUInt64(prevKernelTime);
    uint64_t user = fileTimeToUInt64(userTime) -
fileTimeToUInt64(prevUserTime);

    prevIdleTime = idleTime;
    prevKernelTime = kernelTime;
    prevUserTime = userTime;

    uint64_t totalSys = kernel + user;
    if (totalSys == 0) {
        return 0.0;
    }

    if (idle > totalSys) {
        idle = totalSys;
    }

    double percent = (1.0 - static_cast<double>(idle) /
static_cast<double>(totalSys)) * 100.0;
    if (percent < 0.0) percent = 0.0;
    if (percent > 100.0) percent = 100.0;
    return percent;
}
```

```cpp
SystemMetrics SystemMonitor::getSystemMetrics() {
    SystemMetrics metrics;
    metrics.cpu_percent = calculateCpuUsage();

    MEMORYSTATUSEX memStatus;
    memStatus.dwLength = sizeof(MEMORYSTATUSEX);
    if (GlobalMemoryStatusEx(&memStatus)) {
        metrics.ram_total_mb = static_cast<double>(memStatus.ullTotalPhys)
/ (1024.0 * 1024.0);
        double ram_avail_mb = static_cast<double>(memStatus.ullAvailPhys)
/ (1024.0 * 1024.0);
        metrics.ram_used_mb = metrics.ram_total_mb - ram_avail_mb;
        metrics.ram_percent = static_cast<double>(memStatus.dwMemoryLoad);
    } else {
        metrics.ram_total_mb = 16384.0;
        metrics.ram_used_mb = 8192.0;
        metrics.ram_percent = 50.0;
    }

    // Process count
    int pCount = static_cast<int>(iviProcesses.size());
    HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnap != INVALID_HANDLE_VALUE) {
        PROCESSENTRY32 pe32;
        pe32.dwSize = sizeof(PROCESSENTRY32);
        if (Process32First(hSnap, &pe32)) {
            do {
                pCount++;
            } while (Process32Next(hSnap, &pe32));
        }
        CloseHandle(hSnap);
    }
    metrics.process_count = pCount;
    metrics.uptime_seconds = static_cast<uint64_t>(GetTickCount() / 1000);
    metrics.timestamp = getCurrentIsoTimestamp();

    return metrics;
}

std::string SystemMonitor::priorityClassToString(DWORD priorityClass) {
```

```
    switch (priorityClass) {
        case IDLE_PRIORITY_CLASS: return "IDLE";
        case BELOW_NORMAL_PRIORITY_CLASS: return "BELOW_NORMAL";
        case NORMAL_PRIORITY_CLASS: return "NORMAL";
        case ABOVE_NORMAL_PRIORITY_CLASS: return "ABOVE_NORMAL";
        case HIGH_PRIORITY_CLASS: return "HIGH";
        case REALTIME_PRIORITY_CLASS: return "REALTIME";
        default: return "UNKNOWN";
    }
}

DWORD SystemMonitor::stringToPriorityClass(const std::string& prioStr) {
    std::string s = prioStr;
    std::transform(s.begin(), s.end(), s.begin(), ::toupper);
    if (s == "IDLE") return IDLE_PRIORITY_CLASS;
    if (s == "BELOW_NORMAL") return BELOW_NORMAL_PRIORITY_CLASS;
    if (s == "NORMAL") return NORMAL_PRIORITY_CLASS;
    if (s == "ABOVE_NORMAL") return ABOVE_NORMAL_PRIORITY_CLASS;
    if (s == "HIGH") return HIGH_PRIORITY_CLASS;
    if (s == "REALTIME") return REALTIME_PRIORITY_CLASS;
    return NORMAL_PRIORITY_CLASS;
}

std::vector<ProcessInfo> SystemMonitor::getIviProcessList() {
    return iviProcesses;
}

std::vector<ProcessInfo> SystemMonitor::getProcessList(size_t limit) {
    std::vector<ProcessInfo> list = iviProcesses;

    // Also enumerate host processes to complement the IVI tasks
    HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnap != INVALID_HANDLE_VALUE) {
        PROCESSENTRY32 pe32;
        pe32.dwSize = sizeof(PROCESSENTRY32);

        if (Process32First(hSnap, &pe32)) {
            do {
                if (pe32.th32ProcessID == 0) continue;
```

```
                ProcessInfo info;
                info.pid = pe32.th32ProcessID;
                info.name = pe32.szExeFile;
                info.thread_count = pe32.cntThreads;
                info.memory_mb = 0.0;
                info.priority_class = "NORMAL";
                info.subsystem = "Host OS Service";

                HANDLE hProcess =
OpenProcess(PROCESS_QUERY_LIMITED_INFORMATION | PROCESS_VM_READ, FALSE,
info.pid);
                if (hProcess) {
                    PROCESS_MEMORY_COUNTERS_EX pmc;
                    if (GetProcessMemoryInfo(hProcess,
(PROCESS_MEMORY_COUNTERS*)&pmc, sizeof(pmc))) {
                        info.memory_mb =
static_cast<double>(pmc.WorkingSetSize) / (1024.0 * 1024.0);
                    }
                    DWORD prio = GetPriorityClass(hProcess);
                    if (prio != 0) {
                        info.priority_class = priorityClassToString(prio);
                    }
                    CloseHandle(hProcess);
                }

                list.push_back(info);
            } while (Process32Next(hSnap, &pe32));
        }
        CloseHandle(hSnap);
    }

    if (limit > 0 && list.size() > limit) {
        list.resize(limit);
    }

    return list;
}
```